ÖSTRA REALS GYMNASIUM
Utmaningar – Möjligheter – Mångfald
Vetenskaplig rapport

Naturvetenskapliga programmet inr. Natur
HT22
Kurs: Gymnasiearbete 100 p
Handledare: Johan Skarp, Fredrik Hollsten

Simulation of the evolutionary process

A study of the possibility and use of a simulated evolutionary environment

Namn: Erik Frankling

Klass: Na20E

Abstract

The evolutionary process is the basis for intelligent life and the reason we can create things such as this work. It is paramount that we have a fundamental understanding of this essential process of nature. One method of analyzing this process is simplifying it and simulating it to have a complete overview. Neural networks are a common technology used in simulated evolution because neural networks are simplified representations of animal brains that can be used to evolve advanced behaviors. The purpose of this study is to design an artificial life simulation based on the predator-prey model with individuals capable of evolving behavior. If successful, the simulation should demonstrate that evolution of intelligent behaviors through natural selection in a simulated environment is possible.

An ecosystem simulation consisting of predator, prey and plants was designed. The organisms interact according to simple rules which mimic a real ecosystem. An algorithm called NEAT was implemented to handle the neural network implementation.

Three different simulations were made with varying starting conditions. The results consist of both a visual representation of the organisms' interactions in the simulated ecosystem and diagrams of different data taken from the simulation. The results show the emergence of several different behaviors and that the organisms can adapt to the environment both by responding to the behavior of the other species and by evolving appropriate strategies for the varying environments.

The results of the three simulations conclusively prove that evolution of the organisms behavior have taken place. An extensive analysis of why and how these behaviors emerged could also be done with the help of the graphical representation of the simulation and the different metrics. This shows how simulated evolution can effectively be used to analyze the evolutionary process.

Content

1 INTRODUCTION 4

1.1 Artificial life 4
1.2 Predator-prey model 4
1.3 Artificial neural networks 4
1.4 Topology and weight evolving artificial neural network 6
1.5 Genetic algorithms 6
1.6 Neat 6
1.7 Purpose 7

2 SIMULATION DESIGN (METHOD) 7

2.1 Predator-prey simulation 8
2.2 Prey 9
2.3 Predator 10
2.4 Animal behavior loop 11
2.5 Plants 11
2.6 Neural network implementation 12
2.7 Code structure 13

3 RESULTS 14

3.1 Simulation without predators 14
3.2 Simulation with predators 15
3.3 Simulation with higher move cost 17

4 DISCUSSION/CONCLUSIONS 18

4.1 Population dynamics 18
4.2 Neural network algorithms and learning dynamics 19

SOURCES 19

1 Introduction

1.1 Artificial life

Artificial life is an area of study where simplified artificial systems based on natural living systems are studied to build an understanding of how the natural systems work. The study of artificial life is split into three different areas: simulations, robotic systems and biological systems. This study will focus on simulating an ecosystem with evolutionary mechanisms to understand the functioning of these systems.[1]

1.2 Predator-prey model

A predator-prey model is a model of an ecosystem where two species interact as predator and prey. The model can be analyzed by algebraic methods, setting up equations describing these relationships, or it can be analyzed by simulation methods, using computer simulation. The simulation models the interactions between the two species, and how the population of each species changes over time. The simulation typically includes factors such as predation rates, reproduction rates, and resource availability.[2]

1.3 Artificial neural networks

Artificial neural networks or ANN are computer models inspired by the structure of animal brains. They consist of networks of connected nodes sometimes called artificial neurons that are based on biological neurons. Typically, these networks consist of input nodes that are connected to hidden nodes that perform computation that are connected to output nodes. The connections have corresponding weight values that are multiplied with the value of the connections starting node at the end node. The sum of all the incoming connections and a bias value for a node is transformed through a function such as tanh to ensure an output value within a limited ranges and is then transmitted from the node to the next layer of the network.[3]

[1] Wikipedia, *Artificial life*, https://en.wikipedia.org/wiki/Artificial_life [accessed 2023-01-09]

[2] Wikipedia, *Predator-prey equations*, https://simple.wikipedia.org/wiki/Predatorprey_equations [accessed 2023-01-09]

[3] IBM, *Neural Networks*, https://www.ibm.com/se-en/cloud/learn/neural-networks#:~:text=Neural%20networks%2C%20also%20known%20as,neurons%20signal%20to%20one%20another. [accessed 2023-01-09]

Figure 1: Example of a neural network structure

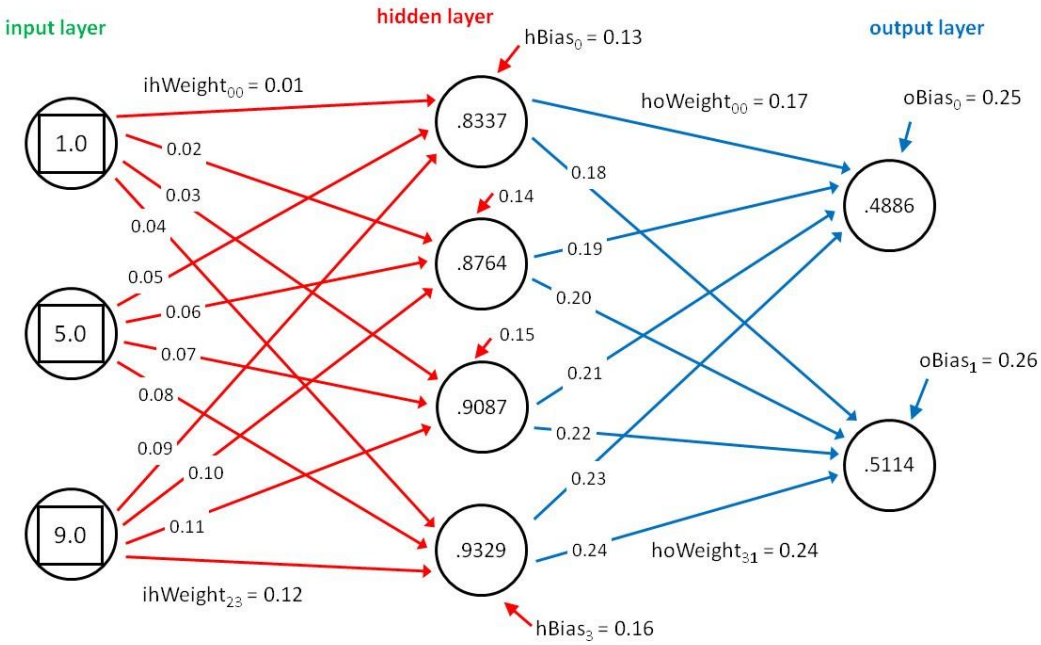
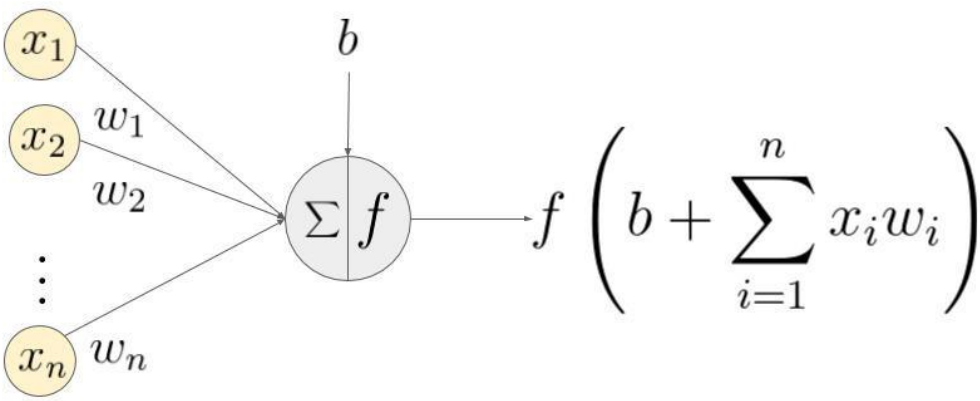


Figure 2: The general description of the computation at each node is



An example of a neuron showing the input (x_1 - x_n), their corresponding weights (w_1 - w_n), a bias (b) and the activation function f applied to the weighted sum of the inputs.

The network is defined by the connections, the weights associated with each connection and by the bias values. In various processes, the networks are evolved by adjusting the connections, the weights, and the bias values. For example, neural networks can be employed to solve optimization problems in an iterative process where the network properties are changed, and the resulting output is evaluated to further improve the network.[4]

[4] IBM, *Neural Networks*

1.4 Topology and weight evolving artificial neural network

The mainstream approach to neural networks is to assume that the network structure is fully connected between the nodes in consecutive layers and fully static, i.e., only weights and biases are adjusted.

Topology and weight evolving artificial neural network or TWEANN are neural networks where both the topology and the weights and biases of a network are evolved to find an optimized network. These networks are less researched than the normal fully connected networks. Generally, TWEANN are efficient at solving simple problems, keeping optimization efforts limited.[5]

1.5 Genetic algorithms

Genetic algorithms or GA are a learning method for neural networks inspired by natural selection. A population of genomes (the connections, weights and biases describing the neural network) is exposed to a test, the outcome of which determines which genomes are used as a basis for further generations of genomes. The outcome is evaluated by a fitness function that ranks the genomes based on their performance. It is possible to not have an explicit fitness function, i.e., when using the neural network as part of a simulation of a natural evolutionary process, where the selection/competition/survival/reproduction process serves as fitness function.[6]

1.6 Neat

Neuro evolution of augmenting topologies or NEAT is a genetic algorithm that improves the learning performance of simple genetic neural network algorithm by introducing specific evolutionary mechanisms:

- Speciation: Protecting innovation by preventing dominant genomes from instantly outcompeting new mutations
- Sexual reproduction: Combining successful genomes to potentially create optimized combined properties
- Minimal starting networks

Historical markings in the genome encoding make it possible to define new species and choose surviving fragments of genomes in sexual reproduction.[7]

Table 1: Comparison of the effects of the NEAT mechanisms, by comparing the number of evaluations required to solve an optimization problem[8]

Method	**Evaluations**
Fixed topology	30239
Non-speciated NEAT	25600
Non-sexual NEAT	5557

[5] Kenneth O. Stanley & Risto Miikkulainen, *Evolving Neural Networks through Augmenting Topologies* (Massachusetts: the MIT press, 2002) s.101

[6] O. Stanley & Miikkulainen 2002 s.101-102
[7] O. Stanley & Miikkulainen 2002 s.106-111
[8] O. Stanley & Miikkulainen 2002 s.117

Full neat	3600

1.7 Purpose

The purpose of this study is to design an artificial life simulation based on the predator-prey model with individuals capable of evolving behavior.

If successful, the simulation should demonstrate that evolution of intelligent behaviors through natural selection in a simulated environment is possible.

1.7.1 Research question

Is it possible to design an artificial life simulation with individuals capable of evolving behavior?

Do the simulated organisms exhibit more intelligent behavior during the simulation?

Does the simulated organism have the ability to adapt to a new environment?

2 Simulation design (Method)

The simulation is implemented in the programming language C#. The game engine Unity is used for graphics and collision detection. There are several parameters that control the simulation, listed in *table 1* below. The explanations for all of them can be found in *table 1*. The parameters have been chosen to yield a stable basic simulation, and some specific changes have been made between the three scenarios studied below.

Table 1: Parameters used to configure the simulation and their values for the three simulations used in the results.

Parameter	Prey only	Predator and prey	Predator and Prey high move cost
Size	125	125	125
Speed	1	1	1
Starting Predators	0	200	200
Starting Prey	400	400	400
Energy Move Cost	0,1	0,1	0,125
Energy Food	45	45	45
Energy Start	25	25	25
Reproduction requirement	70	70	70
Reproduction Cost	60	60	60
Base Energy Cost	0,1	0,1	0,075
Plant Growth Per Tick	0,75	0,75	0,75
Turn Speed	45	45	45
Starting Plants	0,01	0,01	0,01
Speciation Threshold	0,1	0,1	0,1
Add Node Mutation Rate	2	2	2
Add Connection Mutation Rate	15	15	15
Weight Mutation Rate	20	20	20
Remove Connection Rate	1	1	1

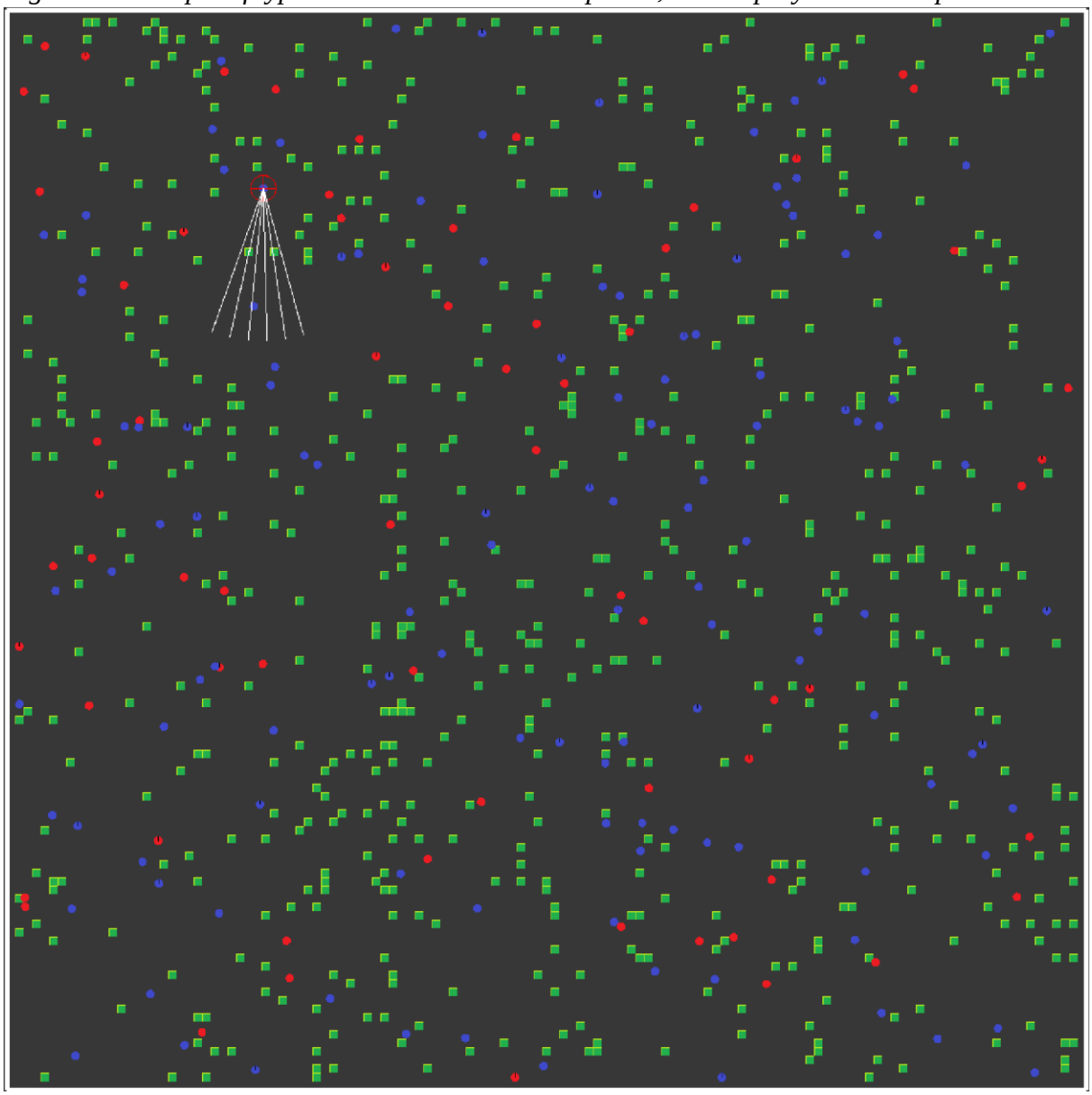
Vision Angel	35	35	35
Vision Angels	6	6	6
Vision Range	18	18	18

When creating a large and complex simulation like this there are a lot of decisions and considerations must be made. Below is a description of how the simulation was designed with some explanations why these design choices where made. However, some decisions that a were deemed to be inconsequential to the results and therefore have no explanation since they are just arbitrary decisions needed to create a functional simulation.

2.1 Predator-prey simulation

The simulation runs in a loop in which every unit of time or tick each animal makes decisions regarding in which direction to move and how fast. In each tick a certain number of plants are created. When a prey animal hits a plant, it consumes it, gaining energy. When a predator and a prey animal meet, the predator eats the prey, gaining energy. At the start of the simulation, a certain number of plants, predators and prey are created. The animals spawn children whenever they have enough energy to do so.

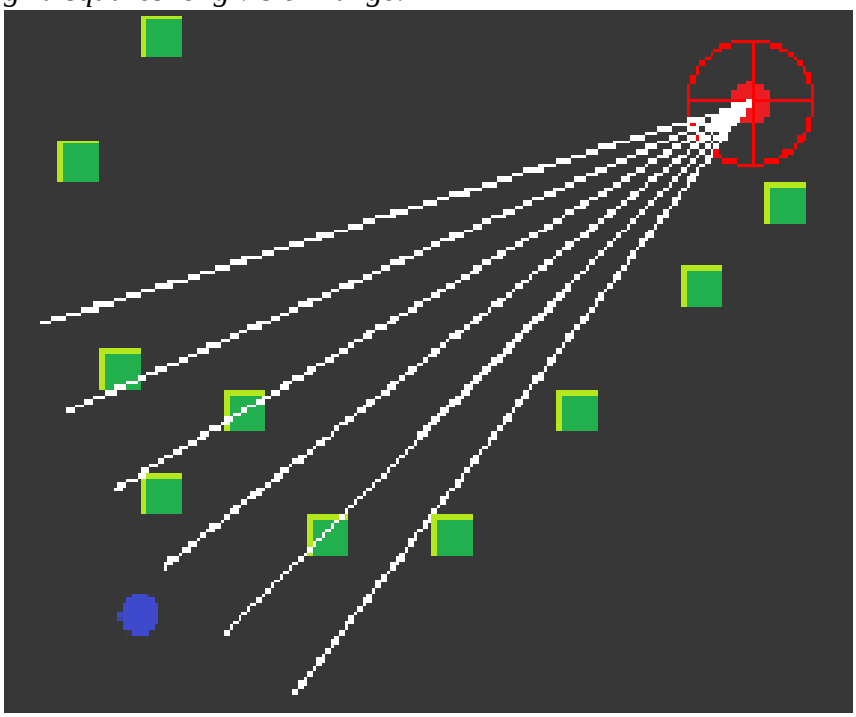
Figure 3: Example of typical simulation. Green is plants, blue is prey and red is predator.



2.2 Prey

Prey are animals and every tick they decide how far to move and how much to rotate. These decisions are made by neural networks with two outputs: how many degrees to rotate to the left or the right; and how far to move, backwards or forwards. The direction change can thus be between -45 (using the default setting in the scenarios, see table above) and +45 degrees. The distance moved can vary between -1 (one grid square backwards) and +1. The input for the neural network is a simulated vision based on several “ray casts” in an arc in front of the animal. A ray cast is a method where a line is drawn from the animal in some direction until it hits something or hit the specified max length. The neural network receives 12 values describing the distance to either a plant or a predator in each of the 6 directions of the ray casts in front of the animal, i.e., six inputs for plants and six inputs for predators seen. Below in Figure 2 is an example of the input rays for an animal.

Figure 4: Graphical representation of inputs for an animal with a 35° arc, 6 inputs and 18 grid squares long vision range.



To survive and reproduce a prey animal must move towards plants and consume them as they receive energy from colliding with a plant. This cannot be achieved standing still as plants do not move. Prey are killed if they collide with predators. Each animal has an individual energy variable that determines if they can reproduce or if they are going to die of starvation. For prey energy is gained from eating plants. Every tick a basic survival energy cost is removed from all animals and an additional movement cost is also removed. The movement cost is multiplied with the speed the animal chooses to move that tick meaning that choosing to sit still consumes less energy but cannot be done for ever as the animal has a base energy cost per tick. Animals also lose energy when reproducing. Reproduction happens when an animal has more energy than the reproduction requirement and then the reproduction cost is removed from the energy level.

When an animal reproduces a new animal is created at a random position in the simulation and with some starting energy. The new animal also gets a new genome created for it; a genome is the 'genetic' information that describes the neural network that the individual animal uses for decision making. The new genome is created by making a copy of the parent network and then potentially applying a mutation to the new genome. Every mutation has a mutation rate in percent which defines how often it occurs in new genomes. Most of the time no mutation occurs, and the new genome is just a copy of the parent genome. The neural network is described further below.

2.3 Predator

Predators are animals like the prey and have essentially the same behavior. Instead of plants, the predators eat prey for energy. The amount of energy gained from consuming a prey is the same as the energy gained by a prey consuming a plant. Predators have no threats and can therefore only die from starvation. Predators have half as many inputs as prey since they only

see prey (and not plants). As predators must consume prey to survive and reproduce, they can either move towards the prey or wait until a prey collides with it.

2.4 Animal behavior loop

The behavior loop that runs every tick for each animal can be summarized as:

- Calculate inputs for the neural network by doing ray casts looking for any relevant objects (for predators: prey; for prey: predators and plants) in several directions in front of the animal.
- Make decisions about how much to move and turn by calculating the outputs of the neural network
- Rotate accordingly
- Move based on the new direction and the decided speed and direction. If the animal hits the edge of the grid the rotation is set to make the animal bounce back from the edge.
- Calculate energy loss this tick by adding the base energy cost and the movement cost multiplied with the movement speed.
- If the energy hits zero, the animal dies of starvation.
- If the animal has more energy than the reproduction requirement subtracts the reproduction cost and create a new animal, subject to a potential failure of the reproduction because of speciation (described below)

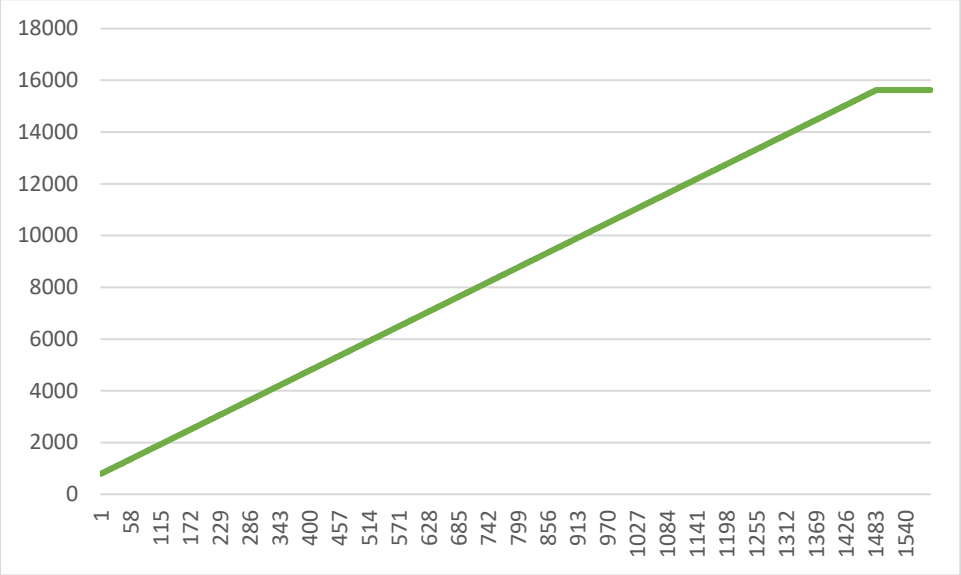
There are two relevant collisions that are relevant for the function of the simulation:

- If the colliders are a plant and a prey the plant is killed, and energy is added to the prey.
- If the colliders are a prey and a predator the prey is killed, and energy is added to the predator.

2.5 Plants

Plants are the only producers in the ecosystem. Only one plant can exist in each 1x1 square of the grid. There is a starting number of plants. Plants are added at a constant rate each tick. Plants are only created in grid squares where no animals are located. In a situation without any animals the plant population follows a linear development as seen below in *figure 5*.

Figure 5: Population of plants in simulation with only plants



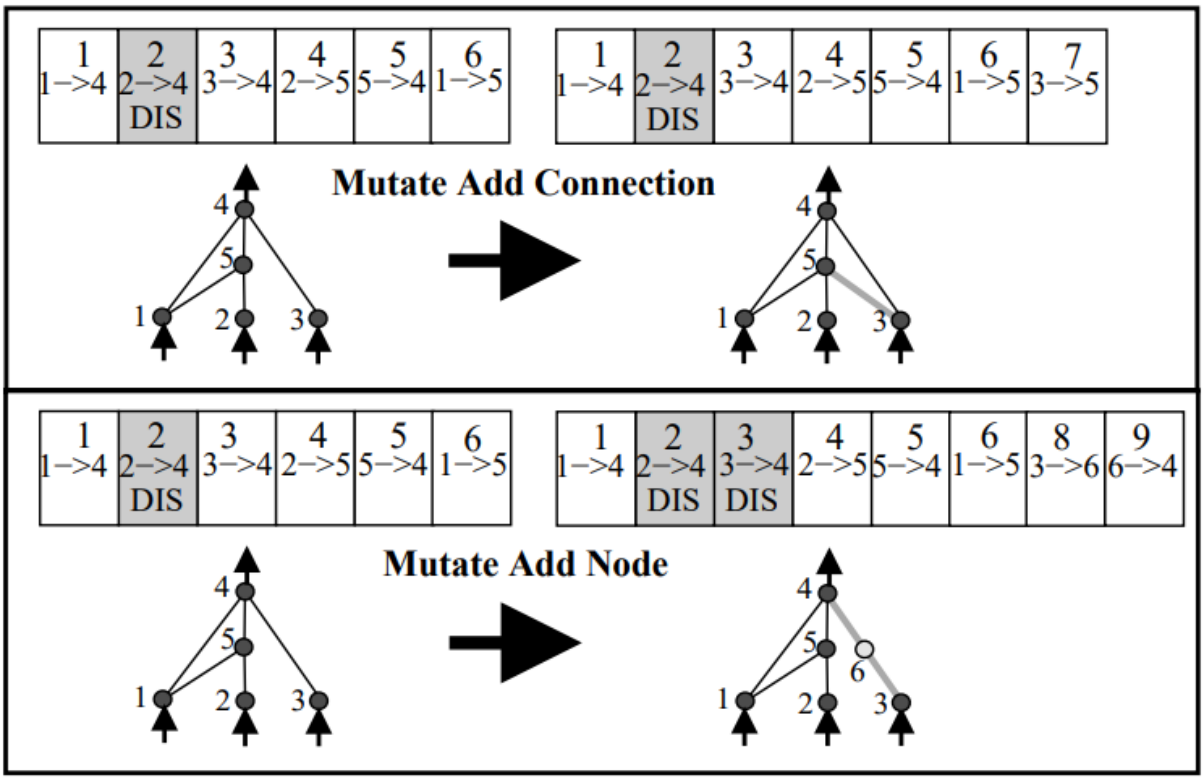
2.6 Neural network implementation

To implement the artificial neural network applying the NEAT algorithm in the predator-prey simulation, each starting animal (predator or prey) is equipped with a starting genome consisting of two output nodes (direction change and speed), six or twelve input nodes for vision and one bias node (with the fixed value of one). The bias node is connected to each of the output nodes, with a weight of zero. This minimal starting genome (with only two connections) is part of implementing the NEAT algorithm. Each time an animal is created (also in the initial generation), mutations occur with certain probabilities, either a change in the weight of the connections between the nodes, a creation of a new connection between nodes (with a random weight), the creation of new node or the removal of a connection or a node. Thus, mutations are necessary to cause the animals to start moving or reacting to vision input.

- The weight mutation picks a random connection and changes its weight up or down by a random amount.
- The add connection mutation picks two random nodes in the genome and creates a new connection between them with a random weight. There are also checks to prevent from creating loops and from making impossible connections like input to input node connection.
- The add node mutation picks a random connection in the genome and replaces it with two new connections with a new node in the middle. The first connection between the previous connections starting node and the new node has the weight 1 and the other new connection from the new node to the previous connections end node is the same as the previous connection. This ensures that the addition of a new node has minimal effect on the animal's behavior since the adding of new structure could otherwise have too big of an effect on the behavior and result in new structure having a hard time emerging. The added node will only have an effect of the behavior if new connections are created, which will potentially combine inputs through the new node.
- The remove connection mutation removes a random connection from the network. After the connection is removed the genome is searched for unnecessary structure

meaning nodes that are either missing input or output connections. These unnecessary structures are then removed meaning that the remove connection mutation also removes nodes. The reason for having a remove mutation is that the fitness landscape in this simulation is dynamic and might require removing old behaviors that were only relevant earlier in the genetic history.

Figure 6: Example diagram of add connection mutation and add node mutation with connection list for reference.



To implement the NEAT speciation principle, the animals are classified into species, depending on the measured genetic difference between each animal's genome and a reference genome for the species. The probability of reproduction failure increases for species that constitute a larger proportion of the animal population.

Sexual reproduction is normally a part of the NEAT algorithm but is not implemented in the simulation.

2.7 Code structure

The simulation C# code consists of the following main classes:

- Controller, running the main loop, controlling the grid and maintaining population statistics and output data
- Organism, base class for predators and prey
- Predator, predator-specific behavior
- Prey, prey-specific behavior
- Plant, plant life cycle

- Genome, managing genetic information for animals, including calculating the outputs of the neural networks, creation of new genomes, mutations etc.
- Nodegene
- ConnectionGene

An instance of the relevant class is created for each animal, plant, genome etc.

The code, data from the simulations used in the results and a executable for running the simulation is available on GitHub: https://github.com/personno2/Simulation-of-the-evolutionary-process

3 Results

Three scenarios have been simulated, as described in *table 1*. Below are charts describing the outcomes. Videos are available on YouTube:

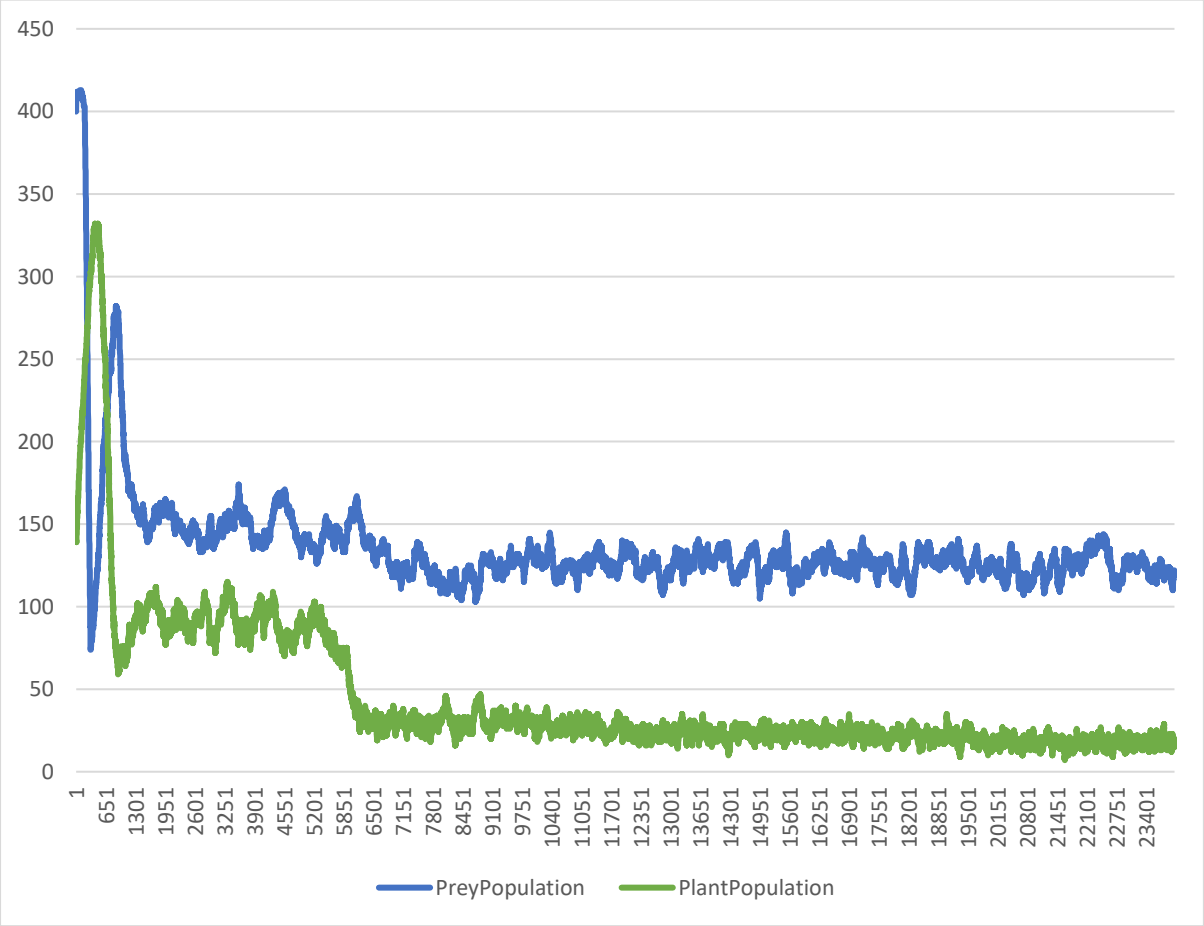
Only prey simulation https://youtu.be/CYvu0VcuyFE

Predator prey simulation https://youtu.be/oHW0YXTkUOw

High move cost simulation https://youtu.be/UVJaWMevd10

3.1 Simulation without predators

Figure 7: Population of plants and prey in simulation without predators.



In the beginning of the simulation only a few preys have the ability to move, and the rest die off quickly. The plant population quickly increases enabling the prey population to expand again. Now all the pray are moving in mostly straight lines at high speed. The prey population stabilizes at the level that the plant growth can sustain (approx. 150 prey), until some prey around 5200 ticks start aiming towards the plants in front of them. Soon all the prey has adopted this behavior. The plant population decreases sharply, as adopt this behavior and more quickly consume the plants that appear. The prey population stays at a similar level. The plant population declines slowly as the prey continue to optimize their movement. The simulation does not change after this until it is turned off.

3.2 Simulation with predators

Figure 8: Population of plants prey and predators.

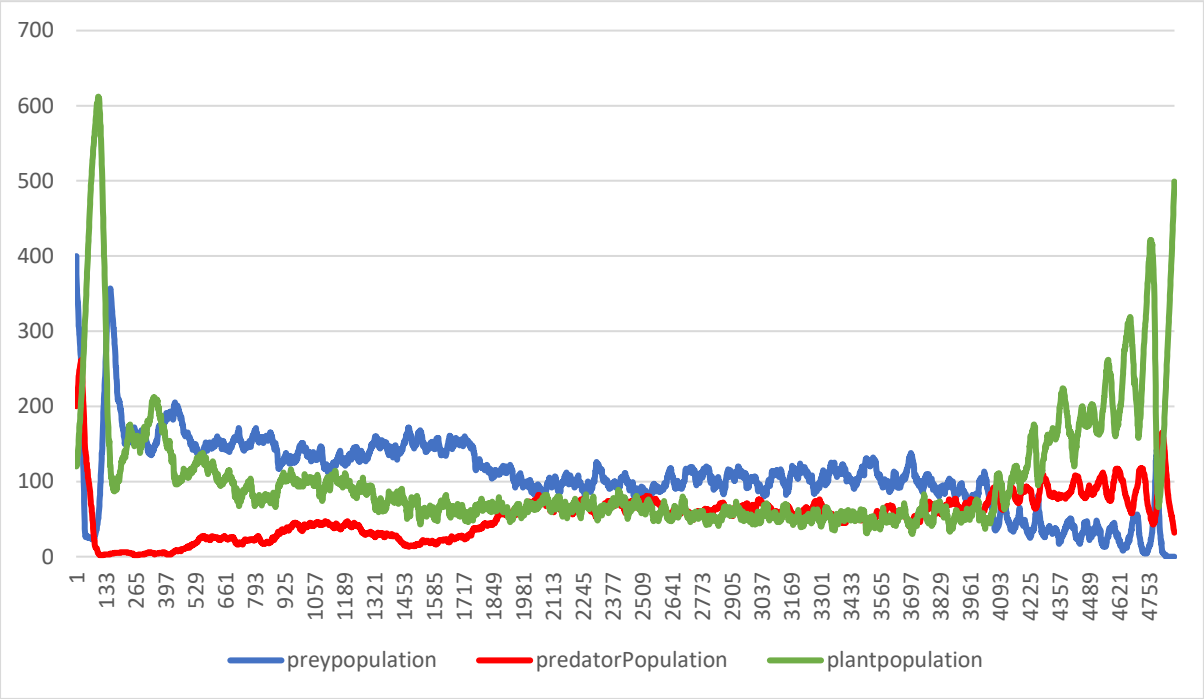
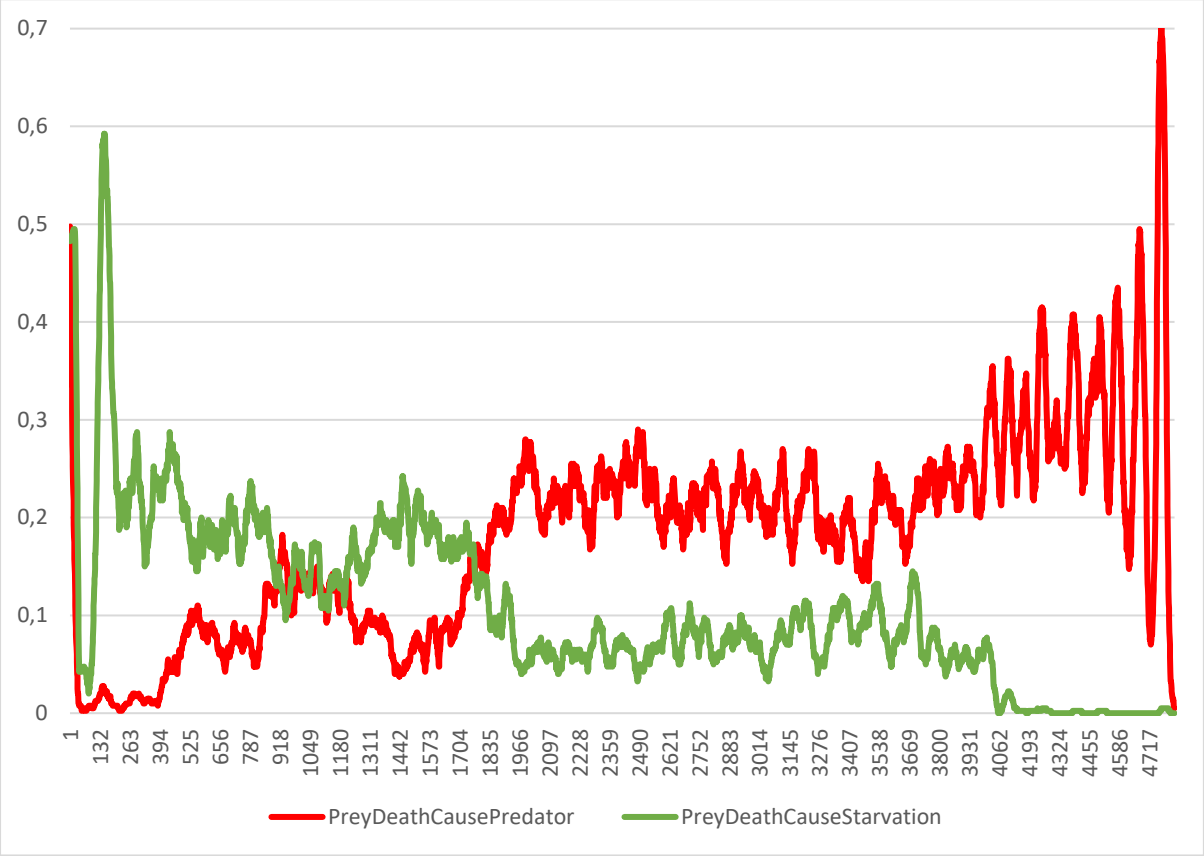


Figure 9: Death causes for prey.



In the beginning most of the animals do not move. All the prey that does not move die off quickly but the few predators that survive both move and remain still, mostly consuming prey that "happen" to collide with predators. The plant population quickly increases and soon the prey population increases in the same way. At this point only one predator is still alive, and it does not move. The large number of preys now mean they start to collide with the predator. Soon there is a decent population of predators again.

The simulation is stable like this for a while. Similarly, to the scenario without predators, the prey improves their ability to aim at the plants more quickly. The small population of predators remain still relying on prey to collide with them. After quite a long time the predators start to learn a new behavior, when there is a prey in front of them the predator moves slightly forward, the predators are still staying still when no prey are nearby, but they appear to be spinning around on the spot.

The predators slowly start to move more and more every time they see a prey and the predators appear to be learning to aim for the pray when moving. Eventually the predator's behavior becomes; sitting still and rotating so the vision ray spins around, when there is a prey in the vision ray, they move at full speed chasing the prey. The predator population increases as they learn this behavior. The prey behavior does not change, and the prey population decreases as the predators increase and in the end the prey is completely extinct.

In the chart of causes of death for prey, the behavioral phases are visible: In the initial phase predators are not the main cause of death for prey; In the middle phase, predators are the major cause of death, as their behavior improves. In the final phase, the predator behavior is

so optimized that they kill off the prey population. The prey does not seem to learn new behavior to avoid the predators.

3.3 Simulation with higher move cost

Figure 10: Population of plants, prey and predators in a simulation with higher move cost.

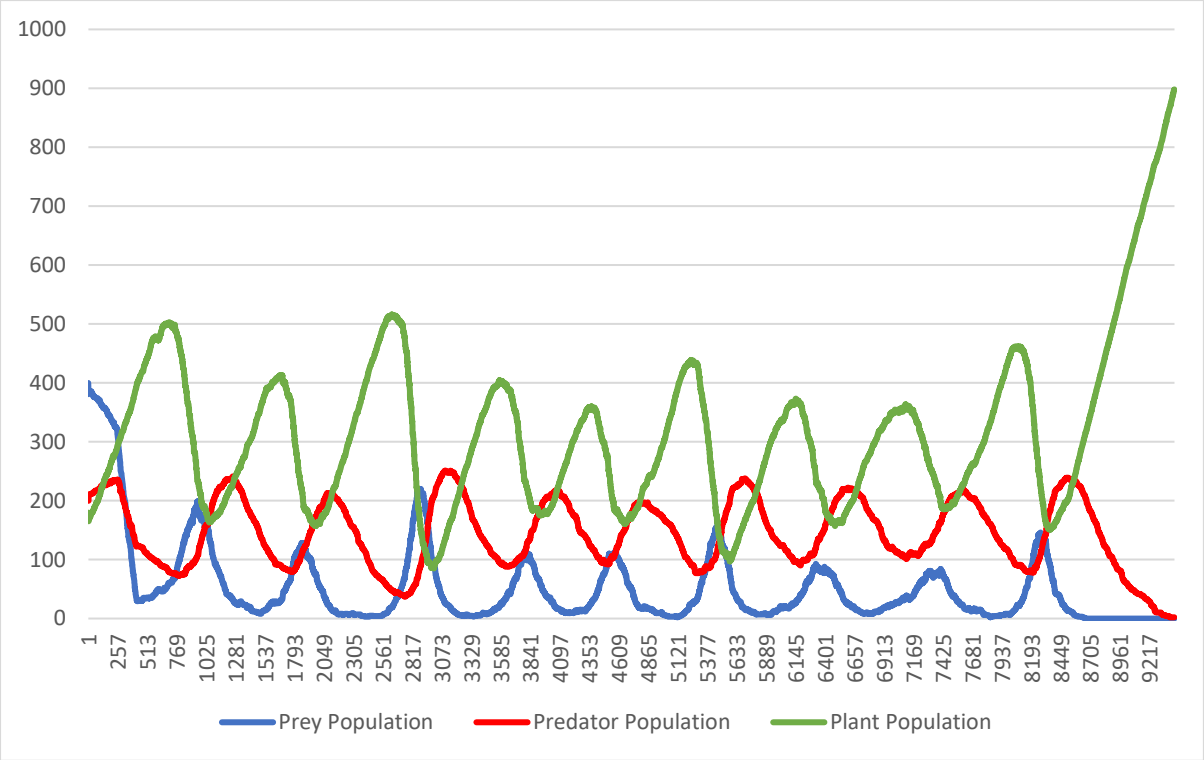
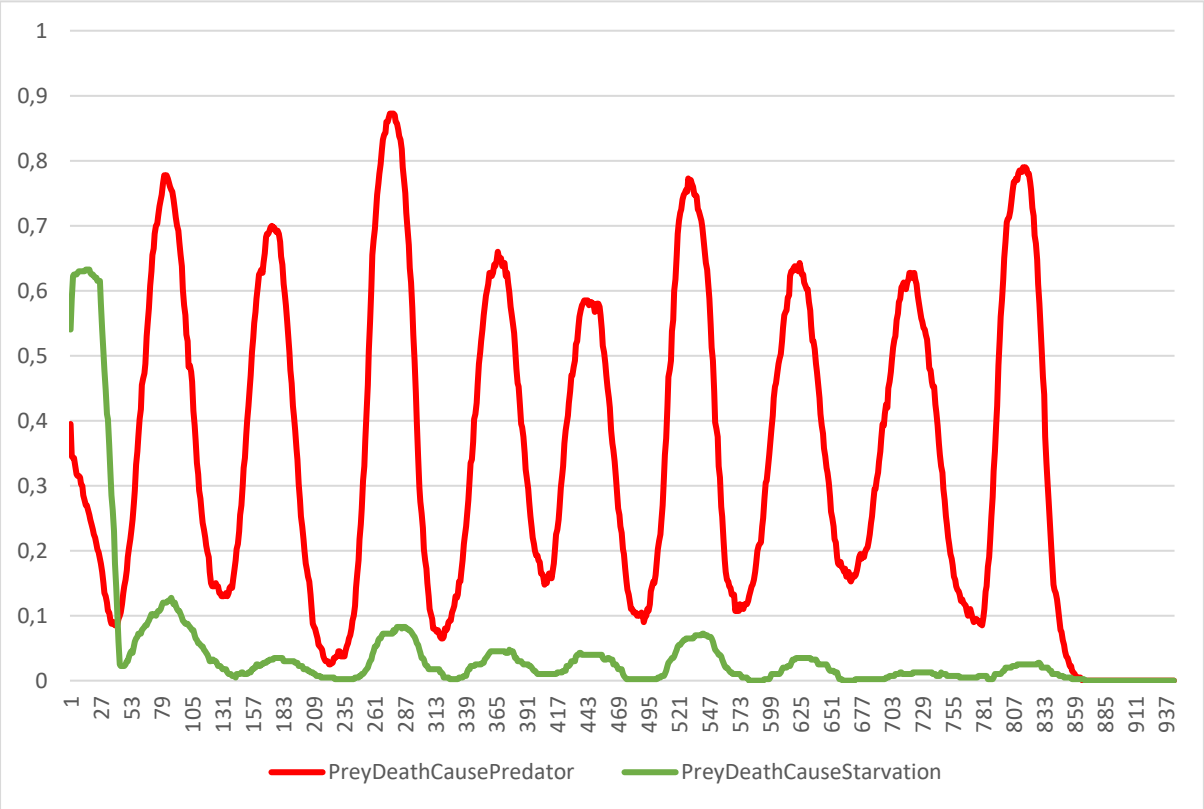


Figure 11: Causes of death for prey in a simulation with higher move cost.



As in the previous scenarios, initially prey that do not move die, reducing the prey population significantly. The predators stay still, as moving is too costly – and unlike the prey, they can get fed without moving as prey collide with them. The population of prey and predators start to oscillate, as growth in prey population with few predators and abundant plants around is strong. With a time, lag, this growth in the prey population enables quick growth of the predator population, which in turn again decimates the prey population. This goes on, without much learning being apparent, until one of the oscillations causes the prey population to completely disappear.

4 Discussion/Conclusions

4.1 Population dynamics

In all scenarios the prey learns to move because all prey that cannot move die of starvation as the plants do not move. This is an example of natural selection working as a learning method because the less fit individuals do not survive and therefore cannot reproduce and spread their genetic material.

Generally, the prey learns to move more quickly. This behavior is easily identified and learned, requiring few neural connections and iterations. The prey individuals that move the most quickly will outcompete the other prey.

Another evolving behavior is the prey learning to aim for plants. This can clearly be seen in the first scenario with only prey as the increased efficiency results in a lower plant population. This behavior is quite complex requiring several mutations to connect several input nodes to outputs and optimizing their weight to turn an appropriate amount in the correct direction.

Even though prey behavior evolved, the sustainable population of prey is always limited by the growth of plants. Only for short periods, prey can consume more than is produced, when they are able to reduce the population of plants, either because they become more efficient in finding the plants (moving from randomly hitting the plants to aiming at them), or because the plant population has grown because the prey population has been temporarily suppressed.

It was more difficult for the predators to learn efficient behavior, beyond the initial choice to not move, as the energy cost of moving did not increase the likelihood of colliding with prey until the predators had learned to aim at prey.

The most advanced behavior that evolved is that of the predators in the second scenario that not only learn to aim at the prey but also learn to stay still and conserve energy while waiting for the next prey.

In the scenario with a higher cost for moving, less ideal conditions for learning can be observed:

- Predators had even stronger incentives to stay still, preventing them from learning to aim
- If the predators stay still, they have a stronger position compared to the prey than in the other scenario, as their energy consumption has been reduced – whereas the prey cannot choose to stay still. This leads to a larger predator population and stronger oscillations in the sizes of the populations, which in turn has two effects on learning:

(i) a very few individuals of prey surviving each cycle, leading to limited genetic diversity and lost learning from cycle to cycle and (ii) the constantly changing conditions for the predators leading to difficulties optimizing between various behavioral patterns – the most efficient behavior with only a couple of prey on the grid is very different from a situation with many prey being available.

4.2 Neural network algorithms and learning dynamics

The simulations show that the neural network genome model works. Simple improvements in the behavior of the animals occur and outcompete less optimized behaviors. However, the simple nature of the network and of the simulated environment led to very simple strategies dominating over more "refined" strategies. For example, the prey did not seem to consider the location of predators in the field of vision.

The implementation of the NEAT algorithm seems to have been efficient. Sexual reproduction was not included, as it is hard to implement in a continuous simulation and its contribution to the learning performance of the NEAT algorithm is relatively small.

The effect of speciation is unclear in the simulation. The implementation relies on making dominant species less successful in reproducing. A possibly better approach to speciation would be making it easier for new species to expand instead of just making it harder for old ones, for example allowing for several children per reproduction or lowering energy requirements. This would however mean favoring a new species just because it is new since increased reproduction is one of the metrics that natural selection optimizes for. It can argue that making this kind of tweaks deviates more from simulating "true" natural selection. The speciation that occurs in nature serves to isolate different genetic lines and can not be quite compared to the NEAT speciation which aims to protect innovation to optimize learning.

Sources

Wikipedia, *Artificial life*, https://en.wikipedia.org/wiki/Artificial_life [accessed 2023-01-09]

Wikipedia, *Predator-prey equations*, https://simple.wikipedia.org/wiki/Predatorprey_equations [accessed 2023-01-09]

IBM, *Neural Networks*, https://www.ibm.com/se-en/cloud/learn/neural-networks#:~:text=Neural%20networks%2C%20also%20known%20as,neurons%20signal%20to%20one%20another. [accessed 2023-01-09]

O. Stanley, Kenneth & Miikkulainen, Risto, *Evolving Neural Networks through Augmenting Topologies* (Massachusetts: the MIT press, 2002) https://nn.cs.utexas.edu/downloads/papers/stanley.ec02.pdf [accessed 2023-01-09]